OPERATING SYSTEMS & NETWORKS · TUTORIAL

Concurrency

Threads, race conditions, locks, condition variables and deadlock — plus the tools that find the bugs you will never find by reading the code.

60 minutes · Three live demos · Clone the code and run along

The hour

Min	What we do	Where
0–10	Threads, and a counter that cannot count	Slides, then a terminal
10–20	Race conditions, and what one cost in 1986	Slides and the board
20–32	Locks and condition variables	Slides
32–45	Producer–consumer, and four ways to break it	Live code
45–57	Deadlock, and how to diagnose a hang	Live gdb
57–60	Tooling, and the checklist to leave with	Slides

Three programs get compiled and run in front of you. All three are broken on purpose — your job is to predict the failure *before* I hit Enter.

Threads: the whole model, in one slide

What a thread is

An independent execution context inside one process: its own program counter, its own registers, its own stack — but running inside the process's single address space. The kernel schedules **threads**, not processes.

Concurrency is not parallelism

Concurrency is about *structure*: several tasks in progress, interleaved in an order nobody promised you. Parallelism is about *execution*: several tasks running in the same instant on different cores. One core gives you concurrency without parallelism — and every bug today still happens.

The trade

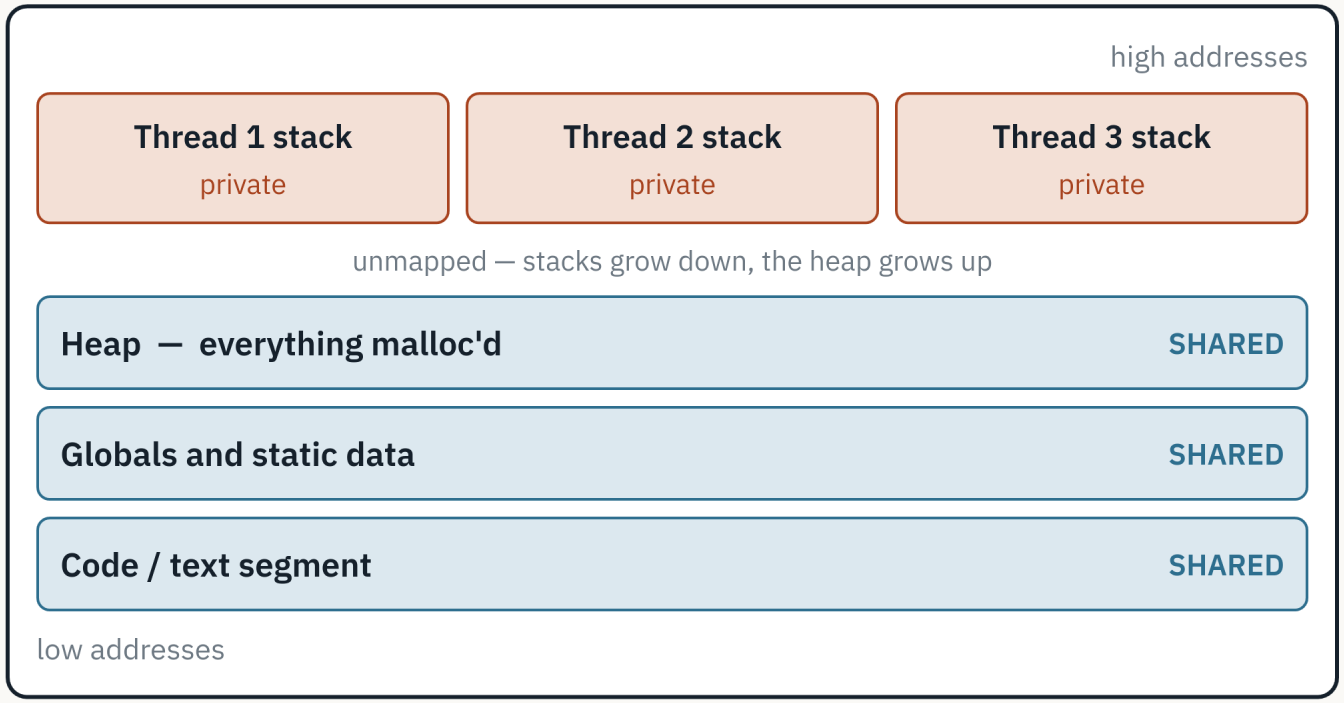
You gain cheap communication through shared memory and real use of multiple cores. You give up isolation: one bad pointer takes down every thread, and all coordination is now your problem.

One process, many threads

Shared by all threads	Private to each thread
Code / text segment	Program counter
Globals and statics	Registers
The heap — everything malloc'd	Its own stack
Open file descriptors	Locals on that stack
Signal handlers, cwd, umask	errno
PID and address space	Signal mask, thread ID

Every row in the left column is a race waiting to happen. That is the entire subject of this tutorial.

One address space, three stacks



Also private — and not in memory at all

- Program counter
- General-purpose registers
- errno (a per-thread location)
- Signal mask, thread ID, priority

A **local variable** lives on the calling thread's own stack. It is safe for free — no other thread can name it.

A **global**, a **static**, or anything behind a pointer into the heap is reachable by every thread. That is what needs a lock.

Where you already rely on this

Every program you use today is multi-threaded. In each case the threads exist so that **one slow task cannot stall the others**.

Web browsers

One thread renders the HTML and CSS of the page in front of you. Another downloads a file in the background. Others run JavaScript and play audio.

Single-threaded: the page freezes until the download finishes.

Web servers

Apache or Tomcat hands each arriving request its own thread, so 1,000 visitors are served at once rather than in a queue.

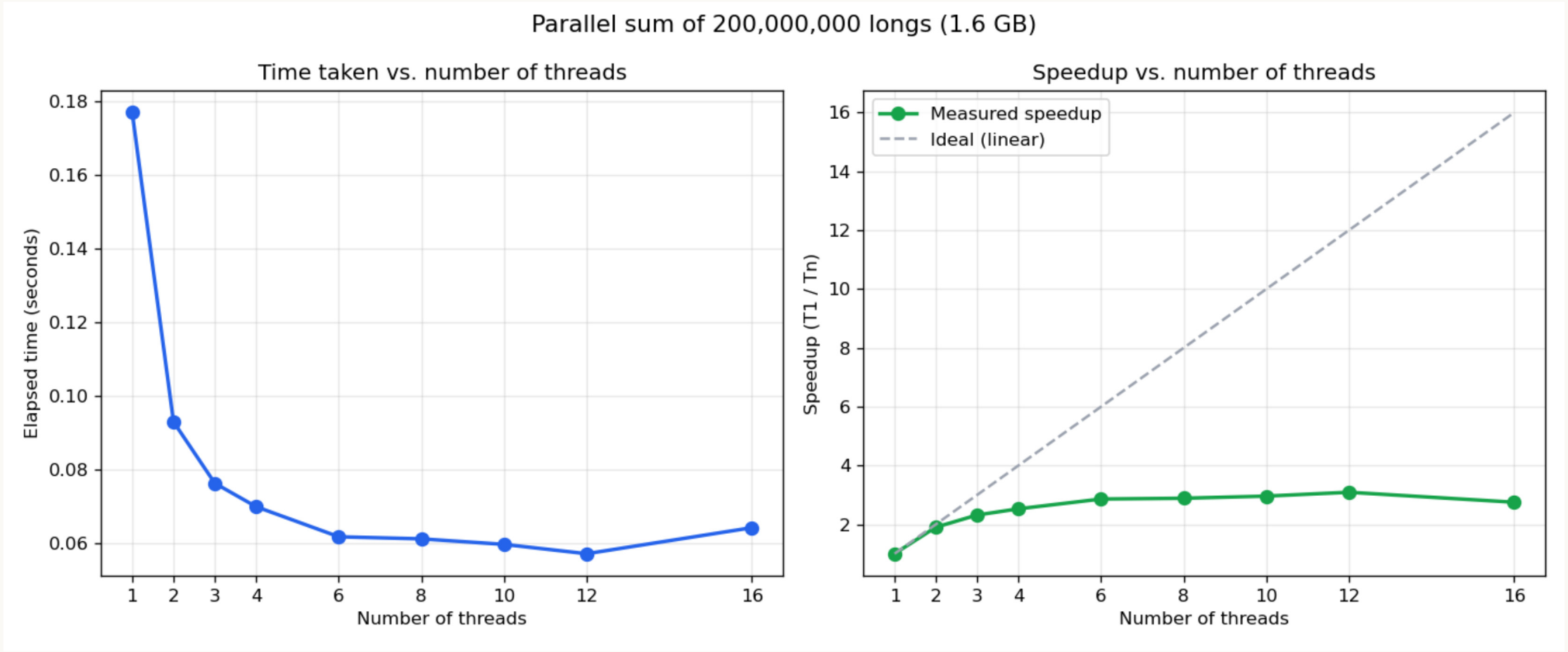
Single-threaded: visitor 1,000 waits behind the other 999.

Video games

Physics, AI pathfinding, audio mixing and render preparation each get a thread, which is how the frame budget is met at 60 or 120 fps.

Single-threaded: one heavy physics step and the picture stutters.

All three share state across those threads — the page's DOM, the server's session cache, the game's world. **Everything that follows in this hour is about that sharing.**


Parallel sum of 200,000,000 longs (1.6 GB)
Time taken vs. number of threads
Elapsed time (seconds)
0.18
0.16
0.14
0.12
0.10
0.08
0.06
1
2
3
4
6
8
10
12
16
Number of threads
Speedup vs. number of threads
Measured speedup
Ideal (linear)
Speedup (T1 / Tn)
16
14
12
10
8
6
4
2
1
2
3
4
6
8
10
12
16
Number of threads

Reference: creating and joining threads in C

```
#include <pthread.h>        /* link with:  gcc -pthread -g prog.c -o prog */



/* ---- create and wait ------------------------------------------------ */
int  pthread_create(pthread_t *tid, const pthread_attr_t *attr,
                    void *(*start_routine)(void *), void *arg);
int  pthread_join(pthread_t tid, void **retval);  /* wait for it to exit  */
int  pthread_detach(pthread_t tid);               /* reaped on its own    */



/* ---- from inside a thread ------------------------------------------- */
void      pthread_exit(void *retval);           /* return a result     */
pthread_t pthread_self(void);                /* my own id           */
int       pthread_equal(pthread_t a, pthread_t b); /* never use == here   */
```

Check the return value

Returns 0, or an errno-style code. These calls do **not** set errno.

Pass arguments carefully

Never hand a thread &i from a loop — they all read the same i.

Join it or detach it

Do neither and its stack is held until the whole process exits.

DEMO 1 · RACE_COUNTER.C

Two threads, one counter, five wrong answers

```
long counter = 0;          /* shared, unguarded */


void *bump(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++;         /* looks atomic */
    return NULL;
}



/* main: spawn 2 threads on bump,

   join both, print counter     */
```

What you will see

Five runs, five different totals. Every one of them **below** 2 000 000 — never above.

Why always below?

Because the only thing that can go wrong is a **lost update**: both threads read the same value, both write back that value plus one. Two increments happened; one of them left no trace.

Before I press Enter: write down the number you expect. Keep it. We will come back to it in five minutes.

```
$ make race_counter && for i in 1 2 3 4 5; do ./race_counter; done
```

A race condition, mechanically

Two threads touch the same memory, at least one of them writes, and **nothing in the program orders the two accesses**. The result then depends on timing that no one promised you.

counter++ is not one operation. It is three:

```
mov    counter(%rip), %rax  # load
add    $1, %rax             # add
mov    %rax, counter(%rip)  # store
```

A thread can be preempted between any two of them. The **critical section** is the stretch from load to store; **mutual exclusion** means only one thread is ever inside it.

One bad interleaving

Thread A	Thread B	memory
load rax = 0		0
	load rax = 0	0
add rax = 1		0
	add rax = 1	0
store 1		1
	store 1	1

Two increments ran. The counter moved by one. Nothing crashed, nothing warned you.

> It passes 999 runs out of 1000. At -O2 it can vanish entirely. **No amount of testing can show you that a race is absent** — only reasoning about the code, or a tool that watches every memory access, can.

CASE STUDY · 1985–1987

Therac-25

The machine

A radiation therapy machine in clinical use from 1983. Two modes: a low-power electron beam, and a high-power X-ray mode in which a thick metal target must first rotate into the beam path to spread it. Fire the high-power beam with no target in place and the patient receives roughly a hundred times the prescribed dose.

The bug

The keyboard-entry task and the turntable-positioning task shared state with no synchronisation. An operator who corrected the mode within about eight seconds — fast enough that only practised typists managed it — left the console showing X-ray mode while the hardware stayed set for electrons. A separate accident traced to a one-byte flag that wrapped to zero, and zero was read as “setup already verified”.

Six known overdoses. Three patients died.

1. **The interleaving was the operator.** It passed testing because the testers typed slowly.
2. **Nondeterminism defeats reproduction.** The manufacturer could not reproduce the fault, and for months held that an overdose was impossible.
3. **The backstop had been removed.** Earlier models carried the same software defects, but hardware interlocks caught them. The 25 trusted the code alone.
4. **The warning was trained away.** “Malfunction 54” was undocumented and routine; operators had learned to clear faults and continue.

Leveson & Turner, “An Investigation of the Therac-25 Accidents”, IEEE Computer 26(7), July 1993

Locks: mutual exclusion, by agreement

What a mutex gives you

At most one thread inside the critical section at a time. It is a **convention, not a barrier** — it works only because every thread that touches the data agrees to take the same mutex first.

A mutex protects data, not code

Write down in a comment which variables each mutex guards. A reader that skips the lock breaks mutual exclusion just as completely as a writer that does.

What is underneath

The lock itself needs an atomic hardware instruction — test-and-set or compare-and-swap. On Linux, a thread that loses the race blocks in a futex syscall instead of spinning forever.

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_init(&m, NULL);   /* or at runtime */

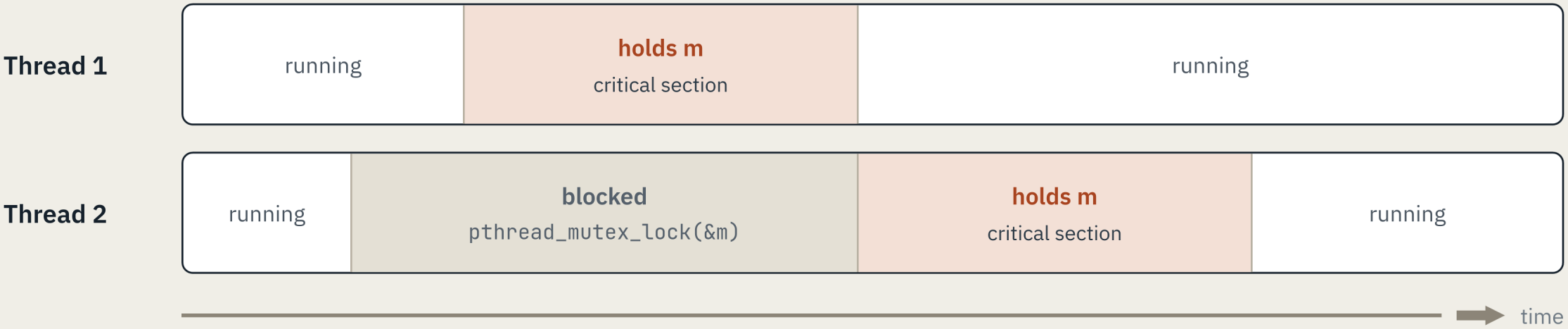

pthread_mutex_lock(&m);         /* blocks        */
pthread_mutex_trylock(&m);      /* EBUSY if held */
pthread_mutex_unlock(&m);
pthread_mutex_destroy(&m);
```

Four rules that prevent most of it

- Take the same mutex for every access — reads too.
- Keep the section short, but never shorter than the invariant.
- Never return or break out while still holding it.
- Never call code you do not control under a lock.

> Also worth knowing: pthread_rwlock_t · sem_t with sem_wait/sem_post · _Atomic long with atomic_fetch_add

What a lock does to the timeline



This is what correct looks like

Thread 2 is not spinning. On Linux it is asleep inside a futex syscall — costing no CPU, and making no progress.

And this is what it cost

For the length of the critical section you have one thread's worth of progress. Widen that block and the two lanes become one.

Three correct fixes. Measure them.

Lock inside the loop

```
for (i = 0; i < N; i++) {
  pthread_mutex_lock(&m);
  counter++;
  pthread_mutex_unlock(&m);
}
```

Correct. Two million lock/unlock pairs, and nearly every one contended.

137 ms

Lock outside the loop

```
pthread_mutex_lock(&m);
for (i = 0; i < N; i++)
  counter++;
pthread_mutex_unlock(&m);
```

Correct, and fastest. Also pointless — thread 2 cannot start until thread 1 has finished.

4 ms

No lock at all

```
_Atomic long counter = 0;

for (i = 0; i < N; i++)
  counter++;
```

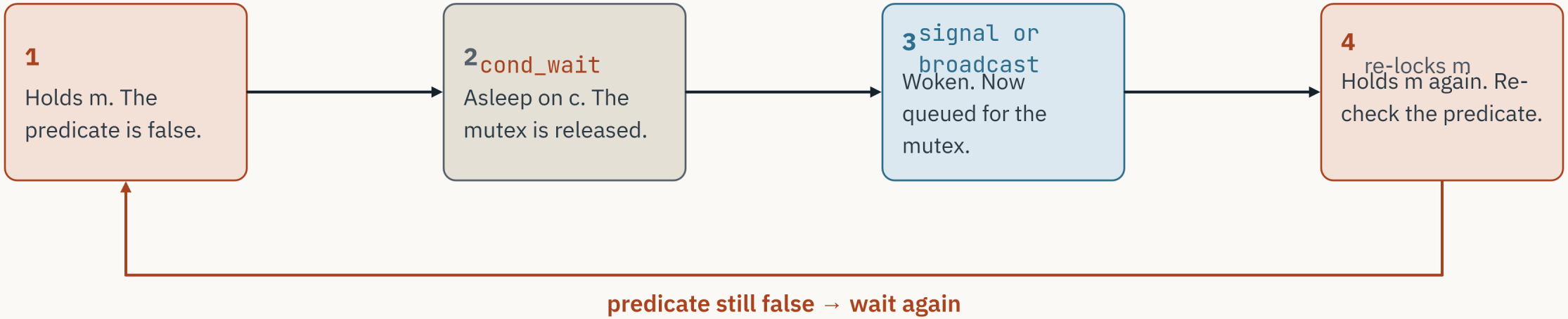
Correct. One locked instruction per increment — no syscall, no queue, no blocking.

28 ms

> All three print 2 000 000 every time. Correctness is settled; what is left is a **granularity** choice. Too coarse and you have written sequential code with extra steps. Too fine and you pay for locking on every operation — and give yourself more lock pairs to get the ordering wrong.

Condition variables: waiting for a predicate

A mutex answers “may I touch this?”. It does not answer “has it become true yet?”. Hold the lock and spin on the predicate and nobody can ever change it; release the lock and re-poll and you burn a core for nothing.



A wake-up is a **hint, not a promise**. Waits can return spuriously, and even after a genuine signal another thread may reach the mutex first and consume what you were waiting for. So the predicate is re-checked every single time: while, never if.

Reference: condition variables, and the one correct shape

The calls

```
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
pthread_cond_init(&c, NULL);


pthread_cond_wait(&c, &m);        /* unlock+sleep */
pthread_cond_timedwait(&c, &m, &deadline);
pthread_cond_signal(&c);          /* wake one     */
pthread_cond_broadcast(&c);       /* wake all     */
pthread_cond_destroy(&c);
```

The only correct shape

```
pthread_mutex_lock(&m);


while (!predicate)          /* while, NOT if */
    pthread_cond_wait(&c, &m);



/* predicate holds, and we hold m */
consume_one_item();


pthread_mutex_unlock(&m);
```

signal or broadcast?

signal wakes at least one waiter; broadcast wakes all of them. If threads waiting on one condvar are waiting for *different* predicates, you need broadcast — or better, give each predicate its own condvar.

The signalling side

Change the state, then signal, both while holding the mutex. Signalling with the lock held is always correct. Signalling after you unlock can be faster and is easier to get wrong.

Producer–consumer: the problem

Producers make items, consumers use them, and a fixed-size buffer sits between so that the two sides need not run at the same rate. It is the canonical exercise because one mutex is not enough — you need mutual exclusion **and** a way to wait.



Buffer full

count == 8. A producer must not overwrite a slot, so it waits until a consumer makes room.

Buffer empty

count == 0. A consumer must not read a slot that was never written, so it waits for an item.

You have already met this: the pipe between two shell commands · a web server's accept loop feeding a worker pool · the NIC ring buffer feeding the kernel · every bounded work queue you will ever write.

The solution: one mutex, two condition variables

```
#define N 8
item_t buf[N];
int count = 0, head = 0, tail = 0;      /* all four guarded by m */
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  not_full, not_empty;     /* count < N  ·  count > 0 */
```

Producer

```
pthread_mutex_lock(&m);

while (count == N)
    pthread_cond_wait(&not_full, &m);
buf[tail] = item;
tail = (tail + 1) % N;
count++;
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);
```

Consumer

```
pthread_mutex_lock(&m);

while (count == 0)
    pthread_cond_wait(&not_empty, &m);
item = buf[head];
head = (head + 1) % N;
count--;
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);
```

Two condvars, not one — so a wakeup meant for a consumer can never be absorbed by a producer.

DEMO 2 · PRODUCER_CONSUMER.C

Four ways to break it

All four compile without a warning. All four survive a short run. Each is a build flag in the demo: three of them fail in front of you, and the fourth refuses to — which turns out to be the most useful one.

```
if (count == 0)
```

Using if instead of while

With two consumers, one signal can let both through. The second finds an empty buffer, reads a slot nothing ever wrote, and drives count to −1.

```
one pthread_cond_t
```

One condvar instead of two

A signal can wake a producer when a consumer was the one that needed to run, and that wakeup is lost. **It did not hang once in 25 runs here** — and that is the lesson. You reason about this one; you cannot test for it.

```
/* no signal */
```

Forgetting to signal

Producers fill the buffer and block. Consumers are never told there is anything to take. Total silence at zero per cent CPU — the clearest hang you will ever see.

```
printf("%d", count)
```

Reading state outside the lock

A progress message that reads count without taking m. Demo 1's race again — this time inside code you believe is synchronised.

```
$ make broken   # pc_broken_if · _onecond · _nosignal · _unlocked
```

Deadlock

All four must hold at once

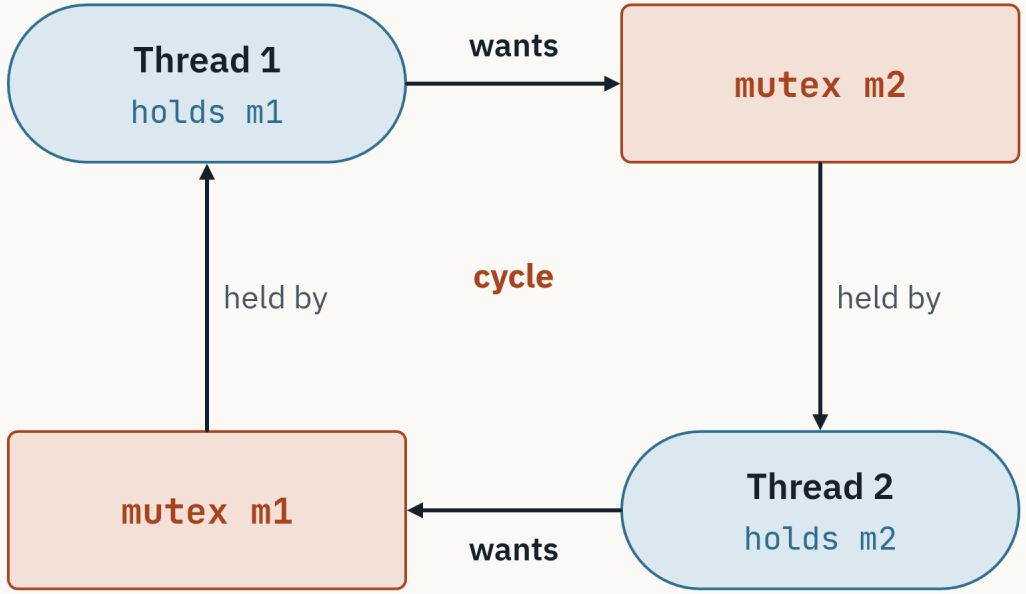
1. **Mutual exclusion** — the resource cannot be shared.
2. **Hold and wait** — a thread holds one and asks for another.
3. **No preemption** — nobody can take a lock away.
4. **Circular wait** — a cycle in the wait-for graph.

Four ways out, one you will use

- **Lock ordering** — kills circular wait.
- **trylock with backoff** — kills hold-and-wait.
- **One lock, or no locks** — avoid it entirely.
- **Detect and recover** — what databases do.

Lock ordering is the only one you can enforce by convention, and it costs nothing at runtime: pick a global order for your mutexes, and always take them in it.

The wait-for graph



> **Not the same thing: livelock** — threads keep running and keep retrying but no one makes progress. **Starvation** — one thread never gets the lock while others keep getting it. Neither shows up as a hang at zero CPU.

DEMO 3 · DEADLOCK.C

Two locks, two orders, one hang

The bug

```
void *t1(void *_) {
    pthread_mutex_lock(&m1);
    usleep(1000);
    pthread_mutex_lock(&m2);  /* hangs */
}
void *t2(void *_) {
    pthread_mutex_lock(&m2);  /* reversed! */
    usleep(1000);
    pthread_mutex_lock(&m1);  /* hangs */
}
```

How to see it

```
$ gdb -p $(pgrep deadlock)

(gdb) thread apply all bt

Thread 2:
#0  __lll_lock_wait ()
#1  pthread_mutex_lock ()
#2  t2 () at deadlock.c:28

Thread 1:
#0  __lll_lock_wait ()
#1  pthread_mutex_lock ()
#2  t1 () at deadlock.c:19
```

The fix: a global order, enforced in one place

```
/* take them in address order, always */
if (a > b) { pthread_mutex_t *t = a; a = b; b = t; }  pthread_mutex_lock(a);  pthread_mutex_lock(b);
```

```
$ make deadlock && ./deadlock &  # then attach
```

The tools that see what you cannot

ThreadSanitizer — start here

```
gcc -fsanitize=thread -g -pthread
```

Instruments every memory access. Reports the race, **both** stack traces, and which lock was held where. Costs 5–15× runtime, and finds in one second what you will not find by reading.

gdb — for a hang

```
gdb -p $(pgrep prog) · thread apply all bt
```

Shows where every thread is parked right now. The only tool for a deadlock, because a deadlocked program has nothing to report — it has simply stopped.

Also	Command	What it finds
Helgrind	valgrind --tool=helgrind ./prog	Races, lock-order inversions, API misuse
AddressSanitizer	gcc -fsanitize=address -g	Corruption a race causes downstream
top / htop	top -H	Deadlock (0% CPU) vs livelock (100%)
strace	strace -f -e trace=futex ./prog	Which futex each thread is blocked on
perf	perf record -g ./prog	Where contention actually costs you
Errorcheck mutex	PTHREAD_MUTEX_ERRORCHECK	Double-locks, unlocking what is not yours

Put a -fsanitize=thread target in your Makefile and run it before every submission.

What to leave with

The checklist

1. Every shared mutable variable is guarded by a named mutex, is atomic, or is immutable — and a comment says which.
2. Every cond_wait sits inside a while on a predicate.
3. You signal the condition you just made true.
4. Every pair of locks has one global order, and every thread takes them in it.
5. The sanitizer runs in your build, not only when something looks wrong.

```
gcc -pthread -g -fsanitize=thread prog.c
gdb -p $(pgrep prog)
(gdb) thread apply all bt
```

Where this goes next

A single-threaded TCP echo server takes one client and starves the second. Thread-per-connection fixes it in about ten lines.

Then it stops scaling at a few thousand connections, because every thread wants its own stack — the one you saw in the address-space figure.

That is the C10K problem, and it is why select, poll and epoll exist. Next tutorial.

If you keep one line from today: **every bug you saw came from two threads sharing memory with no agreement about who goes first.**